

beginning

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

data loading

```
df_encoded = pd.read_excel("/content/drive/MyDrive/capstone/datasets/5_encoded.xlsx")

df_mice_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/6_mice_context.xlsx")
df_mice_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/7_mice_no_context.xlsx")

df_knn_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/8_knn_context.xlsx")
df_knn_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/9_knn_no_context.xlsx")

df_rf_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/10_forest_context.xlsx")
df_rf_no_context = pd.read_excel("/content/drive/MyDrive/capstone/datasets/11_forest_no_context.xlsx")
```

```
def missing_data_report(df):
    # Calculate the percentage of missing data for each column
    missing_percentage = df.isna().mean() * 100

    # Get the data types of each column
    data_types = df.dtypes

    # Create a DataFrame for the missing data summary including data types
    missing_data_summary = pd.DataFrame({
        'Column': missing_percentage.index,
        'Missing Percentage': missing_percentage.values,
        'Data Type': data_types.values
    })

    # Define the four groups based on missing percentage
    group_1 = missing_data_summary[missing_data_summary['Missing Percentage'] > 70][['Column', 'Missing Percentage', 'Data Type']]
    group_2 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 40) & (missing_data_summary['Missing Percentage'] <= 70)][['Column', 'Missing Percentage', 'Data Type']]
    group_3 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 20) & (missing_data_summary['Missing Percentage'] <= 40)][['Column', 'Missing Percentage', 'Data Type']]
    group_4 = missing_data_summary[(missing_data_summary['Missing Percentage'] <= 20)][['Column', 'Missing Percentage', 'Data Type']]
```

```

group_3 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 20) & (missing_data_summary['Missing Percentage'] <= 40)][['Column', 'Missing Percentage', 'Data Type']]
group_4 = missing_data_summary[missing_data_summary['Missing Percentage'] <= 20][['Column', 'Missing Percentage', 'Data Type']]

# Sort each group by missing percentage in descending order (most missing to least missing)
group_1 = group_1.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_2 = group_2.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_3 = group_3.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_4 = group_4.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)

# Rename columns for clarity
group_1.columns = ['Column', 'Missing %', 'Data Type']
group_2.columns = ['Column', 'Missing %', 'Data Type']
group_3.columns = ['Column', 'Missing %', 'Data Type']
group_4.columns = ['Column', 'Missing %', 'Data Type']

# Display each group one by one, from most missing to least missing
print("Group 1: Columns with >70% missing data")
print(group_1.to_string(index=False))
print("\nGroup 2: Columns with >40% to ≤70% missing data")
print(group_2.to_string(index=False))
print("\nGroup 3: Columns with >20% to ≤40% missing data")
print(group_3.to_string(index=False))
print("\nGroup 4: Columns with ≤20% missing data")
print(group_4.to_string(index=False))

```

```
missing_data_report(df_encoded)
```

```

Group 1: Columns with >70% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]
Index: []

```

```

Group 2: Columns with >40% to ≤70% missing data
      Column  Missing %  Data Type
      feels_like  53.061224   float64
      air_humidity  53.061224   float64
      wind_deg  53.061224   float64
      clouds_sky  53.061224   float64
      weather_main_encoded  52.999382   float64
      weather_description_encoded  52.999382   float64
      financial_condition_encoded  45.516388   float64

```

```

Group 3: Columns with >20% to ≤40% missing data
      Column  Missing %  Data Type
workplace_encoded_semi-urban  37.909709   float64
workplace_encoded_urban  37.909709   float64
workplace_encoded_rural  37.909709   float64
      air_pressure  30.426716   float64
      wind_speed  30.179344   float64
      temp_min  29.870130   float64
      temp_max  29.560915   float64
      marital_status_encoded  23.933210   float64

```

```

Group 4: Columns with ≤20% missing data
      Column  Missing %  Data Type
profession_group_encoded  18.923933   float64
      reason_encoded  16.821274   float64
      age  12.739641   float64

```

time_encoded	12.059369	float64
age_group_encoded	7.421150	float64
latitude	6.493506	float64
longitude	6.493506	float64
method_encoded	2.906617	float64
suicide_month	2.411874	float64
suicide_day	2.411874	float64
suicide_weekday_encoded	2.411874	float64
suicide_week	2.411874	float64
suicide_year	2.411874	float64
religion_encoded_buddhist	2.411874	float64
religion_encoded_muslim	2.411874	float64
religion_encoded_indigenous	2.411874	float64
religion_encoded_hindu	2.411874	float64
religion_encoded_christian	2.411874	float64
hometown_encoded	1.669759	float64
gender_encoded_third gender	0.432900	float64
gender_encoded_female	0.432900	float64
gender_encoded_male	0.432900	float64

```
def create_datasets(df_encoded):
    # Columns with >40% to <=70% missing (Group 2)
    group2_columns = [
        'feels_like', 'air_humidity', 'wind_deg', 'clouds_sky',
        'weather_main_encoded', 'weather_description_encoded', 'financial_condition_encoded'
    ]

    # Columns with >20% to <=40% missing (Group 3)
    group3_columns = [
        'workplace_encoded_semi-urban', 'workplace_encoded_urban', 'workplace_encoded_rural',
        'air_pressure', 'wind_speed', 'temp_min', 'temp_max', 'marital_status_encoded'
    ]

    # Columns with <=20% missing (Group 4)
    group4_columns = [
        'profession_group_encoded', 'reason_encoded', 'age', 'time_encoded',
        'age_group_encoded', 'latitude', 'longitude', 'method_encoded',
        'suicide_month', 'suicide_day', 'suicide_weekday_encoded', 'suicide_week',
        'suicide_year', 'religion_encoded_buddhist', 'religion_encoded_muslim',
        'religion_encoded_indigenous', 'religion_encoded_hindu', 'religion_encoded_christian',
        'hometown_encoded', 'gender_encoded_third gender', 'gender_encoded_female',
        'gender_encoded_male'
    ]

    # Dataset 1: Columns with <=20% missing
    df1_columns = group4_columns
    df1 = df_encoded[df1_columns].copy().dropna()

    # Dataset 2: Columns with <=40% missing (Group 3 + Group 4)
    df2_columns = group3_columns + group4_columns
    df2 = df_encoded[df2_columns].copy().dropna()

    # Dataset 3: All columns (Group 2 + Group 3 + Group 4)
    df3_columns = group2_columns + group3_columns + group4_columns
    df3 = df_encoded[df3_columns].copy().dropna()

    return df1, df2, df3
```

```
df_encoded_20, df_encoded_40, df_encoded_70 = create_datasets(df_encoded)
```

```
display(df_encoded_20.shape)
display(df_encoded_40.shape)
display(df_encoded_70.shape)
```

```
(853, 22)
(403, 30)
(57, 37)
```

```
df_mice_context = df_mice_context.dropna()
df_mice_no_context = df_mice_no_context.dropna()
df_knn_context = df_knn_context.dropna()
df_knn_no_context = df_knn_no_context.dropna()
df_rf_context = df_rf_context.dropna()
df_rf_no_context = df_rf_no_context.dropna()
```

```
display(df_mice_context.shape)
display(df_mice_no_context.shape)
display(df_knn_context.shape)
display(df_knn_no_context.shape)
display(df_rf_context.shape)
display(df_rf_no_context.shape)
```

```
(1613, 37)
(1613, 37)
(1617, 37)
(1617, 37)
(1617, 29)
(1617, 29)
```

✓ Leaving the df_encoded_70 dataframe since very low number of rows

```
dataframes = [df_encoded_20, df_encoded_40, df_mice_context, df_mice_no_context, df_knn_context, df_knn_no_context, df_rf_context, df_rf_no_context]
df_names = ["df_encoded_20", "df_encoded_40", "df_mice_context", "df_mice_no_context", "df_knn_context", "df_knn_no_context", "df_rf_context", "df_rf_no_context"]
```

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import DBSCAN
from tqdm import tqdm

# Optional: Uncomment if using HDBSCAN instead of DBSCAN
# !pip install hdbscan
# import hdbscan

def scale_dataframe(df, method='standard'):
    """
    Scale numeric dataframe features.
    method: 'standard' for StandardScaler, 'robust' for RobustScaler
```

```

"""
if method == 'standard':
    scaler = StandardScaler()
else:
    from sklearn.preprocessing import RobustScaler
    scaler = RobustScaler()

df_scaled = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)
return df_scaled

def reduce_dimensions(df, n_components=10, method='pca'):
    """
    Optional dimensionality reduction for clustering.
    method: 'pca' for PCA
    """
    if method == 'pca':
        pca = PCA(n_components=min(n_components, df.shape[1]))
        df_reduced = pd.DataFrame(pca.fit_transform(df))
    else:
        df_reduced = df.copy()

    return df_reduced

def tune_dbscan_for_single_df(df, eps_range=None, min_samples_range=None, metrics_list=None):
    """
    Tune DBSCAN parameters for a single dataframe using combined metric (Silhouette + Calinski + Davies).
    Returns best DBSCAN parameters and cluster labels.
    """
    from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score

    eps_values = eps_range if eps_range is not None else np.arange(0.3, 2.0, 0.1)
    min_samples_values = min_samples_range if min_samples_range is not None else [3, 5, 8, 10, 15]
    metrics_values = metrics_list if metrics_list is not None else ['euclidean', 'manhattan', 'cosine']

    X = df.values

    best_combined_score = -1
    best_labels = None
    best_params = None

    total_combinations = len(eps_values) * len(min_samples_values) * len(metrics_values)
    with tqdm(total=total_combinations, desc="Tuning DBSCAN") as pbar:
        for eps in eps_values:
            for min_samples in min_samples_values:
                for metric in metrics_values:
                    try:
                        labels = DBSCAN(eps=eps, min_samples=min_samples, metric=metric).fit_predict(X)
                        n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
                        if n_clusters < 2:
                            pbar.update(1)
                            continue
                        sil_score = silhouette_score(X, labels)
                        ch_score = calinski_harabasz_score(X, labels)
                        db_score = davies_bouldin_score(X, labels)
                        # Normalize and combine metrics
                        ch_norm = ch_score / (ch_score + 1e-6)

```

```

        db_norm = 1 / (1 + db_score)
        combined_score = (sil_score + ch_norm + db_norm) / 3
        if combined_score > best_combined_score:
            best_combined_score = combined_score
            best_labels = labels
            best_params = {'eps': eps, 'min_samples': min_samples, 'metric': metric}
    except:
        pass
    pbar.update(1)

return best_params, best_labels

def dbscan_clustering_pipeline(df, scale=True, reduce_dim=True, n_components=10,
                              eps_range=None, min_samples_range=None, metrics_list=None):
    """
    Full sequential clustering pipeline for a single dataframe.
    Steps:
    1. Scaling (optional)
    2. Dimensionality reduction (optional)
    3. DBSCAN clustering with parameter tuning

    Returns:
    df_copy with 'dbscan_cluster' column added.
    """
    df_copy = df.copy()

    # Step 1: Scale
    if scale:
        df_scaled = scale_dataframe(df_copy)
    else:
        df_scaled = df_copy.copy()

    # Step 2: Dimensionality Reduction
    if reduce_dim:
        df_reduced = reduce_dimensions(df_scaled, n_components=n_components)
    else:
        df_reduced = df_scaled.copy()

    # Step 3: Tune DBSCAN and get best cluster labels
    best_params, best_labels = tune_dbscan_for_single_df(df_reduced,
                                                         eps_range=eps_range,
                                                         min_samples_range=min_samples_range,
                                                         metrics_list=metrics_list)

    df_copy['dbscan_cluster'] = best_labels
    return df_copy

```

```

# Loop through each dataframe
for df in dataframes:
    # Run the pipeline and modify the dataframe in-place
    df_clustered = dbscan_clustering_pipeline(df, scale=True, reduce_dim=True, n_components=10)

    # Copy the new cluster labels into the original dataframe
    df['dbscan_cluster'] = df_clustered['dbscan_cluster']

```

```

Tuning DBSCAN: 100%|██████████| 255/255 [00:08<00:00, 30.30it/s]
Tuning DBSCAN: 100%|██████████| 255/255 [00:01<00:00, 136.97it/s]
Tuning DBSCAN: 100%|██████████| 255/255 [00:16<00:00, 15.27it/s]
Tuning DBSCAN: 100%|██████████| 255/255 [00:19<00:00, 13.08it/s]
Tuning DBSCAN: 100%|██████████| 255/255 [00:17<00:00, 14.72it/s]
Tuning DBSCAN: 100%|██████████| 255/255 [00:18<00:00, 14.09it/s]
Tuning DBSCAN: 100%|██████████| 255/255 [00:15<00:00, 16.90it/s]
Tuning DBSCAN: 100%|██████████| 255/255 [00:16<00:00, 15.75it/s]

```

```

from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
from scipy.spatial.distance import cdist
import numpy as np

```

```

def evaluate_clusters(
    df,
    label_col,
    ignore_label=None,      # e.g., -1 for DBSCAN, None for KMeans/Agglo/KNN
    sil_metric="euclidean", # match your clustering distance
    sample_size=None,
    random_state=42,
    return_dict=False
):
    """
    General-purpose clustering evaluation function.
    Works for Agglomerative, DBSCAN, KMeans, Spectral, GMM, etc.
    """

    rng = np.random.RandomState(random_state)

    X = df.drop(columns=[label_col]).values
    labels = df[label_col].values

    # Optionally ignore a label (e.g., DBSCAN noise = -1)
    if ignore_label is not None:
        mask = labels != ignore_label
        X, labels = X[mask], labels[mask]

    # Optional subsample for speed (applies to all metrics)
    if sample_size is not None and X.shape[0] > sample_size:
        idx = rng.choice(X.shape[0], size=sample_size, replace=False)
        X, labels = X[idx], labels[idx]

    unique_clusters = np.unique(labels)
    n_clusters = len(unique_clusters)

    if n_clusters <= 1:
        print("⚠ Only one cluster found. Metrics cannot be computed robustly.")
        return {} if return_dict else None

    results = {}

    # Core indices
    try:
        results["Silhouette Score"] = {
            "score": silhouette_score(X, labels, metric=sil_metric),
            "range": "[-1, 1]",

```

```

        "better": "higher"
    }
except Exception:
    results["Silhouette Score"] = {"score": np.nan, "range": "[-1, 1]", "better": "higher"}

try:
    results["Calinski-Harabasz Index"] = {
        "score": calinski_harabasz_score(X, labels),
        "range": "[0, ∞)",
        "better": "higher"
    }
except Exception:
    results["Calinski-Harabasz Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "higher"}

try:
    results["Davies-Bouldin Index"] = {
        "score": davies_bouldin_score(X, labels),
        "range": "[0, ∞)",
        "better": "lower"
    }
except Exception:
    results["Davies-Bouldin Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "lower"}

# Dunn Index (guard against singletons / zero intra-distances)
def dunn_index(X, labels):
    eps = 1e-12
    uniq = np.unique(labels)
    # Intra-cluster diameters
    intra = []
    for k in uniq:
        pts = X[labels == k]
        if pts.shape[0] >= 2:
            d = cdist(pts, pts)
            intra.append(np.max(d))
        else:
            intra.append(0.0) # singleton cluster
    denom = max(max(intra), eps)

    # Inter-cluster minimum distance
    inter_min = np.inf
    for i in range(len(uniq)):
        for j in range(i + 1, len(uniq)):
            Xi = X[labels == uniq[i]]
            Xj = X[labels == uniq[j]]
            d = cdist(Xi, Xj)
            inter_min = min(inter_min, np.min(d))
    if not np.isfinite(inter_min):
        return np.nan
    return inter_min / denom

# Xie-Beni Index (guard against identical centroids)
def xie_beni_index(X, labels):
    eps = 1e-12
    uniq = np.unique(labels)
    centroids = np.array([X[labels == k].mean(axis=0) for k in uniq])
    # Compactness

```



```

compact = 0.0
for i, k in enumerate(uniq):
    pts = X[labels == k]
    diff = pts - centroids[i]
    compact += np.sum(diff * diff)
# Min centroid separation
min_csep = np.inf
for i in range(len(centroids)):
    for j in range(i + 1, len(centroids)):
        d = np.sum((centroids[i] - centroids[j]) ** 2)
        if d < min_csep:
            min_csep = d
if not np.isfinite(min_csep) or min_csep <= eps:
    return np.nan
return compact / (X.shape[0] * min_csep)

try:
    results["Dunn Index"] = {
        "score": dunn_index(X, labels),
        "range": "[0, ∞)",
        "better": "higher"
    }
except Exception:
    results["Dunn Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "higher"}

try:
    results["Xie-Beni Index"] = {
        "score": xie_beni_index(X, labels),
        "range": "[0, ∞)",
        "better": "lower"
    }
except Exception:
    results["Xie-Beni Index"] = {"score": np.nan, "range": "[0, ∞)", "better": "lower"}

# Pretty print
print("\n📊 Clustering Evaluation Results:")
for metric, info in results.items():
    score = info['score']
    score_str = f"{score:.4f}" if isinstance(score, (int, float)) and np.isfinite(score) else "NaN"
    print(f"{metric}: {score_str} ----- Range: {info['range']} ----- {info['better'].capitalize()} is better")

return results if return_dict else None

```

```

for df, name in zip(dataframes, df_names):
    print("\n-----")
    print(f"Evaluation metrics for {name}: total clusters (incl. noise): {df['dbscan_cluster'].nunique()}")
    print("=====")

    results = evaluate_clusters(
        df,
        label_col='dbscan_cluster',
        ignore_label=-1,      # important for DBSCAN
        sil_metric="euclidean",
        sample_size=None,     # full dataset (no subsampling)
        return_dict=True
    )

```

)

```
-----
Evaluation metrics for df_encoded_20: total clusters (incl. noise): 2
=====

 Clustering Evaluation Results:
Silhouette Score: 0.1219 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 1.6904 ----- Range: [0, ∞) ----- Higher is better
Davies-Bouldin Index: 2.5161 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 0.0718 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 1.7644 ----- Range: [0, ∞) ----- Lower is better

-----
Evaluation metrics for df_encoded_40: total clusters (incl. noise): 3
=====

 Clustering Evaluation Results:
Silhouette Score: -0.3391 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 8.7399 ----- Range: [0, ∞) ----- Higher is better
Davies-Bouldin Index: 1.3885 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 0.0089 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 1.6224 ----- Range: [0, ∞) ----- Lower is better

-----
Evaluation metrics for df_mice_context: total clusters (incl. noise): 3
=====

 Clustering Evaluation Results:
Silhouette Score: 0.8744 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 96.5472 ----- Range: [0, ∞) ----- Higher is better
Davies-Bouldin Index: 0.1313 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 2.3277 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 0.0104 ----- Range: [0, ∞) ----- Lower is better

-----
Evaluation metrics for df_mice_no_context: total clusters (incl. noise): 2
=====

 Clustering Evaluation Results:
Silhouette Score: -0.0779 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 2.3449 ----- Range: [0, ∞) ----- Higher is better
Davies-Bouldin Index: 2.5485 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 0.0100 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 2.5461 ----- Range: [0, ∞) ----- Lower is better

-----
Evaluation metrics for df_knn_context: total clusters (incl. noise): 46
=====

 Clustering Evaluation Results:
Silhouette Score: -0.4050 ----- Range: [-1, 1] ----- Higher is better
Calinski-Harabasz Index: 220.0613 ----- Range: [0, ∞) ----- Higher is better
Davies-Bouldin Index: 4.7182 ----- Range: [0, ∞) ----- Lower is better
Dunn Index: 0.0034 ----- Range: [0, ∞) ----- Higher is better
Xie-Beni Index: 130.3311 ----- Range: [0, ∞) ----- Lower is better

-----
Evaluation metrics for df_knn_no_context: total clusters (incl. noise): 43
```

```

from google.colab import files
import pandas as pd

# Assuming dataframes and df_names are already defined and populated
# from previous cells.

output_dir = "/content/dbscan_exported_dfs"
os.makedirs(output_dir, exist_ok=True)

exported_files = []

for df, name in zip(dataframes, df_names):
    # Construct the output filename
    output_filename = f"dbscan_{name}.xlsx"
    output_filepath = os.path.join(output_dir, output_filename)

    # Export the dataframe to an Excel file
    try:
        df.to_excel(output_filepath, index=False)
        print(f"Exported {name} to {output_filepath}")
        exported_files.append(output_filepath)
    except Exception as e:
        print(f"Error exporting {name}: {e}")

# Provide a way to download the files
if exported_files:
    print("\nYour files are ready for download:")
    for f in exported_files:
        print(f)
    # You can manually download from the file browser in Colab,
    # or use the following code to download them programmatically:
    # for f in exported_files:
    #     files.download(f)
else:
    print("\nNo files were exported.")

Exported df_encoded_20 to /content/dbscan_exported_dfs/dbscan_df_encoded_20.xlsx
Exported df_encoded_40 to /content/dbscan_exported_dfs/dbscan_df_encoded_40.xlsx
Exported df_mice_context to /content/dbscan_exported_dfs/dbscan_df_mice_context.xlsx
Exported df_mice_no_context to /content/dbscan_exported_dfs/dbscan_df_mice_no_context.xlsx
Exported df_knn_context to /content/dbscan_exported_dfs/dbscan_df_knn_context.xlsx
Exported df_knn_no_context to /content/dbscan_exported_dfs/dbscan_df_knn_no_context.xlsx
Exported df_rf_context to /content/dbscan_exported_dfs/dbscan_df_rf_context.xlsx
Exported df_rf_no_context to /content/dbscan_exported_dfs/dbscan_df_rf_no_context.xlsx

Your files are ready for download:
/content/dbscan_exported_dfs/dbscan_df_encoded_20.xlsx
/content/dbscan_exported_dfs/dbscan_df_encoded_40.xlsx
/content/dbscan_exported_dfs/dbscan_df_mice_context.xlsx
/content/dbscan_exported_dfs/dbscan_df_mice_no_context.xlsx
/content/dbscan_exported_dfs/dbscan_df_knn_context.xlsx
/content/dbscan_exported_dfs/dbscan_df_knn_no_context.xlsx
/content/dbscan_exported_dfs/dbscan_df_rf_context.xlsx
/content/dbscan_exported_dfs/dbscan_df_rf_no_context.xlsx

```

